ChatGPT

Interactive Brokers Web API – Comprehensive Documentation (IBKR Campus)

Version: Documentation content as of December 2025.

Table of Contents

- **Part I: IBKR Web API Documentation (Overview & Usage)**
- Introduction
- Feedback
- Getting Started
  - Web API Access for Organizations
  - Web API Access for Individuals
  - Web API Access for Third Parties
- Usage and Support
  - Contacting API Support
  - Scheduled Server Maintenance
  - Weekday IServer Reset Timing
  - Pacing Limitations
  - Per-Endpoint Request Rate Limits
  - Additional Usage Limits
- Sessions in the Web API
- Instrument Discovery
- Market Data
  - Top-of-Book Snapshots
  - Streaming Top-of-Book Data
- Orders
  - New Order Example
  - Order Reply Messages
  - Order Reply Suppression
  - Order Rejections (placeholder)
  - Previewing Orders (placeholder)
  - Modifying Orders
  - Canceling Orders
  - Submitting Bracket Orders (placeholder)
  - Orders for Combos/Spreads
  - Monitoring Live Orders
  - Monitoring Executions (placeholder)
- Portfolio and Positions
  - Querying Your Accounts
  - Querying Currency Balances
  - Querying Equity and Margin
- Advisor Features (placeholder)

• FYIs, Alerts, and Bulletins

- Types of Notification Messages (FYIs, Alerts, Bulletins)
- FYIs
- Unread FYIs
- All Recent FYIs
- Marking FYIs Read
- Managing FYI Subscriptions
- FYI Disclaimers
- FYI Delivery Options (TBD)
- Alerts
- Creating and Modifying Alerts
- Listing Configured Alerts
- Retrieving Alert Details
- Mobile Trading Alerts
- Bulletins

• **Part II: IBKR Web API Reference (Beta)**

• Note on Unified API Reference (Beta)

• Legacy Documentation Links

- Trading (Client Portal) Web API
- Account Management Web API
- Flex Web Service API

• **Part III: Client Portal Web API v1.0 Documentation (Detailed Reference)**

• Introduction
• Requirements & Limitations
- Canadian Residents Restrictions

• Web API Basics Tutorial
• Client Portal Gateway
- Limitations of the Gateway
- How to Download and Run the Gateway
- Download and install Java Runtime Environment
- Download and unzip the API Gateway
- Run the API Gateway
- How to Modify the Gateway Port
- Client Portal Gateway FAQ

• Authentication
- Session Authentication
- Background: CP Web API Sessions
- Understanding Brokerage Sessions
- Understanding IBKR Terminology
- Managing Multiple Sessions
- Using a Paper Account
- Authentication FAQ

• Pacing Limitations
• Regular Server Maintenance (Reset Times)

- Endpoints (REST API)
    - **Alerts**: Create/Modify Alert; List Alerts; Alert Details; MTA Alert Info; Activate/Deactivate Alert; Delete Alert
    - **Accounts**: Account Profit and Loss; Search Dynamic Account; Set Dynamic Account; Signatures and Owners; Switch Account; Receive Brokerage Accounts
    - **Contract**: (Multiple endpoints for contract and securities lookup; e.g. SecDef search, contract info by conid, etc.)
    - **Event Contracts**: (Categorization; Markets & Strikes; Order Submission; Executions & Netting)
    - **FA Allocation Management**: (Sub-Accounts; List/Modify/Delete Allocation Groups; Allocation Presets; etc.)
    - **FYIs and Notifications**: (Unread Bulletins; Subscription List; Enable/Disable Subscription; FYI Typecodes; FYI Disclaimer; Delivery Options; List Notifications; Mark Notification Read)
    - **Market Data**: (Live Market Data Snapshot; Update Frequency; Regulatory Snapshot; Market Data Availability; Historical Data; Unsubscribe streams)
    - **Option Chains**: (Instantiate Option Chain; Find Strikes; Validate Contract; etc.)
    - **Order Monitoring**: (Live Orders; Order Status; Trades)
    - **Orders**: (Place Order; Order Reply Confirmation; Respond to Server Prompt; Preview/WhatIf Order; Overnight Order; Bracket & OCA Orders; Combo/Spread Orders; Cancel Order; Modify Order; Suppress Messages; Suppressible MessageIds; Reset Suppressed Messages)
    - **Portfolio**: (Portfolio Accounts; Portfolio Subaccounts; Positions; Combination Positions; Portfolio Summary; Portfolio Ledger; Position & Contract Info)
    - **PortfolioAnalyst**: (All Periods; Account Performance; Transaction History)
    - **Scanner**: (IServer Scanner Params; IServer Market Scanner; Historical Data Scanner)
    - **Session**: (Authentication Status; Initialize Brokerage Session; Initialize HMDS; Logout; Ping; Validate SSO)
    - **Watchlists**: (Create Watchlist; Get All Watchlists; Get Watchlist Details; Delete Watchlist)

- WebSockets API

    - Connection Guide
    - Request Session Information
    - Retrieve the Session Token
    - Establishing WebSocket (with Gateway vs OAuth)
    - Sending WebSocket Topics
    - Solicited vs Unsolicited Messages
    - Subscribing to Topics
    - Account Operations (Account Summary/Ledger subscription management)
    - Market Data (Subscribe/Cancel market data streams; Historical data stream)
    - Misc Operations (Exercise Options; Ping Session)
    - Order & Position Operations (Live Order updates; PnL updates; Trades updates)
    - Unsolicited Message Types (e.g. Account Updates, etc.)

- **Part IV: Web API Changelog (Chronological)**

Part I: IBKR Web API Documentation (Overview & Usage)

Introduction

Interactive Brokers is merging our web-based API products into a single, comprehensive **IBKR Web API**, bringing the features of the Client Portal Web API, Digital Account Management API, and the Flex Web Service together in a unified interface, accessible by a shared means of authorization and authentication: **OAuth 2.0** [1]. Existing endpoints and authentication schemes are **not deprecated** and will continue to receive updates; rather, IBKR is providing a new coordinated set of endpoints exposing the same backend resources [2]. The documentation below addresses functionality of the Client Portal Web API, Digital Account Management, and Flex Web Service side-by-side under the Web API umbrella [3].

IBKR's development resources are now organized into two sections [4]:

- **Documentation:** Long-form, workflow-oriented material (this section).
- **Reference:** Per-endpoint API definitions, presented in a Swagger interface (some endpoints may still be pending addition).

Feedback

Have feedback on our Web API documentation or reference material? Email us at **API-Feedback@interactivebrokers.com**. Suggestions and feedback help us continuously improve our API solutions [5]. *(Note: This is an automated inbox; for specific support questions, contact the API Support team.)*

Getting Started

Clients with fully open and funded accounts may use the Web API immediately without any special onboarding or approval process [6]. However, the Web API offers several methods of authentication, some of which do require approval or configuration before use [7]. Additionally, certain feature sets (e.g. Account Registration, Funds & Banking) are not available to all account types and require configuration by IBKR's Sales Engineering team prior to use [8].

Web API Access for Organizations

Enterprise and Institutional clients have several authorization methods available. All methods allow direct requests to IB's infrastructure [9]. The Web API consists of two broad feature sets: **Trading** and **Account Management**. Trading functionality is available to all accounts, while some Account Management features may not apply to certain account structures and require special approval/onboarding [10]. Contact your IBKR Sales Representative for guidance on the onboarding process [11].

Available access methods for organizations include [12]:

- **OAuth 2.0 (beta):** Supports first-party (own accounts) and third-party (unaffiliated client accounts, with consent) access. Offers access to account management and trading features.
- **OAuth 1.0a:** Supports first-party and third-party usage (with client authorization). Offers access to trading features **only**.
- **SSO (Single Sign-On):** Available to Financial Advisors and Introducing Brokers. Supports development of alternative UIs for client accounts under your management. Offers access to account management and trading features.

Web API Access for Individuals

Individual client use of the Web API involves a standard IBKR username/password login. The live account must be fully open, funded, and of type "IBKR Pro" (even if using its paper account) [13]. If you do not have an account, you can **create one for free** [14] (note: account must be activated before API use).

Web API Access for Third Parties

IBKR defines third-party developers as vendors of software that interacts with IB client accounts to which the vendor has no formal relationship. (By contrast, Advisors or Brokers formally manage client accounts.) Third-party vendors currently may seek approval **only for OAuth 1.0a** usage [15]. Such vendors must receive compliance approval for their product offerings before integration. The approval process involves:

1. Initial screening by IB's onboarding team (approx. 2–3 weeks). If passed, the application goes to Compliance [16].
2. IBKR Compliance conducts due diligence and a multi-tier review (approx. 3–6 weeks) [17].
3. If Compliance approves, IBKR Legal provides a Web API agreement for review/signature. Meanwhile, the onboarding team requests public keys and a callback URL to configure the OAuth1.0a consumer (detailed instructions provided at that stage; approx. 3–5 weeks) [18].

Timelines are estimates and can vary. Third-party applicants should provide as much information as possible upfront to avoid delays [19]. Compliance expects vendors to have a complete website detailing their product, including clear workflows and functionality descriptions [20]. Significant changes to the offering after approval (e.g. adding trading functionality) would require additional Compliance review [21]. Third-party vendors offering automated trading solutions are expected to hold appropriate regulatory registrations in all regions they serve (or provide legal rationale if not) [22]. Compliance approval is needed from all regions where IBKR clients are served [23].

Usage and Support

Contacting API Support

Interactive Brokers welcomes users to contact customer support for API issues. To expedite troubleshooting, consider the following first [24]:

- Does the issue persist on other platforms (e.g. does the same order error occur in Trader Workstation)?
- Did the request include all **Required** parameters as per documentation?
- Were relevant guides or documentation consulted prior to reaching out?
- Is the error new or has the request worked previously (if so, how long)?
- Do similar requests still work as expected?

IBKR offers multiple support channels (accessible via the Client Portal): **Support Ticket**, **Live Chat with API Support**, and **Phone Support** [25]. Use the appropriate channel as needed.

Scheduled Server Maintenance

IBKR conducts scheduled maintenance on the infrastructure serving the Web API. During maintenance windows, some Web API features may be temporarily unavailable [26]. Note that the Web API's maintenance timing can differ from other platforms like TWS [27].

- **Uptime:** The Web API is generally available 24 hours/day on weekdays, with maintenance only on Saturday evenings [28].
- **Brokerage Session Resets:** Brokerage functionality (the `/iserver` endpoints) is briefly unavailable each evening around 01:00 local time by region [29].

Weekday IServer Reset Timing

Region	Maintenance Onset (Local Time)
North America (NY/Chicago)	01:00 US/Eastern [30]
Europe	01:00 CEST [31]
Asia	01:00 HKT [32]

Pacing Limitations

IBKR enforces a **global rate limit** of **50 requests per second** for each authenticated user session (username) [33]. Users connecting via the Client Portal Gateway tool have a lower limit of **10 requests per second** [33]. Exceeding these limits results in HTTP **429 Too Many Requests** errors [34]. Violating IPs may be penalized: a 10-minute "penalty box" blocking period, with repeated offenses potentially leading to a permanent block until resolved [35].

Per-Endpoint Request Rate Limits

Some endpoints have their own stricter pacing limits, as listed below (requests beyond these will also trigger 429 errors) [36]:

```
Endpoint                                  | Method | Limit
------------------------------------------|--------|---------------------
/iserver/marketdata/snapshot              | GET    | 10 requests / second
/iserver/scanner/params                   | GET    | 1 request / 15 mins
/iserver/scanner/run                      | POST   | 1 request / second
/iserver/trades                           | GET    | 1 request / 5 secs
/iserver/orders                           | GET    | 1 request / 5 secs
/iserver/account/pnl/partitioned          | GET    | 1 request / 5 secs
/portfolio/accounts                       | GET    | 1 request / 5 secs
/portfolio/subaccounts                    | GET    | 1 request / 5 secs
/pa/performance                           | POST   | 1 request / 15 mins
/pa/summary                               | POST   | 1 request / 15 mins
/pa/transactions                          | POST   | 1 request / 15 mins
/fyi/unreadnumber                         | GET    | 1 request / second
/fyi/settings                             | GET    | 1 request / second
/fyi/settings/{typecode}                  | POST   | 1 request / second
/fyi/disclaimer/{typecode}                | GET    | 1 request / second
/fyi/disclaimer/{typecode}                | PUT    | 1 request / second
```

```
/fyi/deliveryoptions                      | GET    | 1 request / second
/fyi/deliveryoptions/email                | PUT    | 1 request / second
/fyi/deliveryoptions/device               | POST   | 1 request / second
/fyi/deliveryoptions/{deviceId}           | DELETE | 1 request / second
/fyi/notifications                        | GET    | 1 request / second
/fyi/notifications/more                   | GET    | 1 request / second
/fyi/notifications/{notificationId}       | PUT    | 1 request / second
/tickle                                   | GET    | 1 request / second
/sso/validate                             | GET    | 1 request / minute
```

Additional Usage Limits

Other usage limits include [37]:

- **/trsv/secdef** (POST): max **200 conids per request**.
- **/iserver/marketdata/history** (GET): max **5 concurrent requests**.

Sessions in the Web API

Accessing **Trading** functionality via Web API requires establishing a **trading-enabled brokerage session** [38]. A brokerage session is tied to an IBKR username (which has trading permissions for one or more accounts) [39]. A given username can have only one active brokerage session at a time across all IBKR platforms [40]. Trading permissions, market data subscriptions, etc., are tied to usernames (not accounts), so references to "brokerage session" mean a logged-in username connected to IBKR's backend trading infrastructure [41].

Although only one brokerage session per username is allowed, some Web API features are accessible without a brokerage session. This means a user's active brokerage session (e.g. in TWS) can continue undisturbed while certain **read-only** API calls are made [42]. We refer to these as a "read-only" subset of Trading API features (e.g. portfolio retrieval, instrument search) available without `/iserver` endpoints. Thus, **Web API sessions are two-tiered** [43]:

1. An **outer read-only session** (always required to use any Client Portal Web API request). By itself it allows only non-`/iserver` endpoints (no trading).
2. The **brokerage session**, established after the read-only session, which permits access to trading, market data, and other `/iserver` endpoints.

In practice, this mirrors how the Client Portal web interface works: if your username already has an active trading session elsewhere, you can log into Client Portal in a restricted mode for account management (read-only) without disturbing the trading session [42]. When logging in fresh via API, you start with a read-only session and then initialize the brokerage session (via an endpoint) to enable trading features [44].

Instrument Discovery

Documentation coming soon. (No current content in this section.)

Market Data

To retrieve top-of-book, depth-of-book, or historical market data via the Web API, you must have [45] :

- An IBKR username with the appropriate **live market data subscriptions** and trading permissions for the instruments.
- An **authorized Web API session** (logged in).
- A **brokerage session** (i.e. access to `/iserver` endpoints, if requesting streaming or trading-related data).

Top-of-Book Snapshots

Top-of-book snapshot requests return up-to-date market data values (bid/ask, last price, volume, etc.) from the same streams displayed in TWS watchlists [46] .

Values needed: to request a snapshot, you need [47] :

- The **Contract ID** (`conid`) for each instrument of interest.
- The **Market Data field tags** (`fields`) you want (each tag represents a specific market data point). A comprehensive list of available field tags is in the Web API Reference material [48] .

A `GET` request to the `/iserver/marketdata/snapshot` endpoint is used for snapshots. Required query params [49] :

- `conids` – comma-separated list of instrument conids.
- `fields` – comma-separated list of field tags.

Snapshot Pre-Flight: Before data is returned for a given `conid`, you must make an initial "pre-flight" request to instruct IB's server (IServer) to begin streaming that instrument's market data [50] . The first call for new conids will return no data, only echoing the conids to confirm the stream is initiated [51] . Example:

```
GET https://api.ibkr.com/v1/api/iserver/marketdata/snapshot?
conids=265598,8314&fields=31,7059,84,88,86,85
```

If it's the first time requesting snapshots for conids 265598 and 8314, the response will be:

```
[
  { "conid": 265598, "conidEx": "265598" },
  { "conid": 8314,   "conidEx": "8314" }
]
```

This indicates IServer is now streaming those instruments. After this, subsequent snapshot requests for those conids (even without specifying fields again) will return the data [52] . For example:

```
GET https://api.ibkr.com/v1/api/iserver/marketdata/snapshot?
conids=265598,8314
```

The response might be:

```
[
  {
    "31": "168.42",
    "6119": "q1",
    "6509": "RpB",
    "7059": "100",
    "84": "168.41",
    "85": "600",
    "86": "168.42",
    "88": "1,300",
    "_updated": 1712596911593,
    "conid": 265598,
    "conidEx": "265598",
    "server_id": "q1"
  },
  {
    "31": "189.60",
    "6119": "q2",
    "6509": "RpB",
    "7059": "100",
    "84": "189.56",
    "85": "500",
    "86": "189.61",
    "88": "200",
    "_updated": 1712596911593,
    "conid": 8314,
    "conidEx": "8314",
    "server_id": "q2"
  }
]
```

Some fields (especially those computed at intervals) may not show up until they have an updated value [53].

Streaming Top-of-Book Data

To open a **live** top-of-book market data stream, you send a message over the WebSocket connection in the format [54]:

```
smd+<CONID>+{"fields":["field1","field2",...,"fieldN"]}
```

The `fields` values are the same numeric tags used in the HTTP snapshot request, but they must be given as strings in the JSON array [55]. For example, to stream IBM (conid 8314) fields 31,84,85,86,88,7059:

```
smd+8314+{"fields":["31","84","85","86","88","7059"]}
```

If successful, the WebSocket will start receiving messages like [56] :

```
{
  "31": "189.60",
  "6119": "q2",
  "6509": "RpB",
  "7059": "100",
  "84": "189.56",
  "85": "500",
  "86": "189.61",
  "88": "200",
  "_updated": 1712596911593,
  "conid": 8314,
  "conidEx": "8314",
  "server_id": "q2",
  "topic": "smd+8314"
}
```

To cancel a streaming market data topic, send a message: `umd+<CONID>+{}`. For example, to cancel the above stream:

```
umd+8314+{}
```

Orders

All order-related functionality assumes you have [57] :

- An IBKR username with appropriate trading permissions.
- An authorized Web API session.
- A brokerage session (access to `/iserver` endpoints).
- The Account ID of an account (under that username) to place the order in.

New Order Example

The following is a workflow for submitting a **new order** via the Web API [58] .

Values needed: [59]

- **Contract ID (`conid`)** for the instrument.
- **Order handling instructions** (order parameters such as type, side, quantity, etc.).

A `POST` to `/iserver/account/{accountId}/orders` creates a new order ticket for the specified account `{accountId}` [60] . Path parameter:

- `{accountId}` – The account ID in which to place the order.

This endpoint also requires a JSON body. The exact keys depend on order type and other factors (see Order Types documentation), but **at minimum**, every order JSON must include [61] :

- `conid` – the instrument's contract ID.
- `orderType` – the order type (e.g. "LMT" for limit, "MKT" for market, etc.).
- `side` – side of the trade ("BUY" or "SELL").
- `tif` – time in force (e.g. "DAY" for day order, "GTC" for good-till-cancel, etc.).
- `quantity` – number of units to trade.

Note: Keys and values are case-sensitive, and correct JSON types must be used (consult Reference for all available keys) [62] [63] .

Example: Suppose account **DU123456** has trading permissions. We want to buy 100 shares of AAPL at a limit price of \$165, day order [64] . We know:

- AAPL (US stock in USD) has conid `265598` [65] .
- We need to represent our instructions as JSON:
- Buy order: `"side": "BUY"` [66] .
- Quantity 100: `"quantity": 100` [66] .
- Limit order: `"orderType": "LMT"` [66] .
- Price \$165: `"price": 165` [66] .
- Day order: `"tif": "DAY"` [66] .
- Conid: `"conid": 265598` [67] .

Combining these:

```
POST https://api.ibkr.com/v1/api/iserver/account/DU123456/orders
[
  {
    "conid": 265598,
    "side": "BUY",
    "orderType": "LMT",
    "price": 165,
    "quantity": 100,
    "tif": "DAY"
  }
]
```

Note: The body is a JSON **array** containing the order object. The array format allows submitting bracket orders (multiple related orders) in one call, but for a single order, just one object in the array [68] .

If successful, the response will include an `order_id` and `order_status` (among possibly other fields) indicating the order was received [69] . For example:

```
{
  "order_id": "987654",
  "order_status": "Submitted",
```

```
  "encrypt_message": "1"
}
```

Order Reply Messages

Sometimes an order submission does **not** immediately return an acknowledgement, but instead returns an **"order reply" message** that requires confirmation before the order will be submitted [70]. This does **not** mean the order is rejected; rather, IBKR requires you to explicitly confirm some aspect of the order (often due to precautionary settings or "fat finger" protections) [71].

For example, a reply message might warn that the order violates a user-defined percentage price constraint and ask for confirmation [72]:

```
[
  {
    "id": "07a13a5a-4a48-44a5-bb25-5ab37b79186c",
    "message": [
      "The following order \"BUY 100 AAPL NASDAQ.NMS @ 165.0\" price
exceeds\nthe Percentage constraint of 3%.\nAre you sure you want to submit
this order?"
    ],
    "isSuppressed": false,
    "messageIds": [
      "o163"
    ]
  }
]
```

Important fields in such a reply [73]:

- `id` – a unique identifier for this message. You will use this in a follow-up call to confirm or cancel the message.
- `messageIds` – an array of codes categorizing the type of message (here `"o163"`). These codes can be used to **suppress** similar messages for the rest of the session (see *Order Reply Suppression*).

To confirm and proceed with the order, call the `/iserver/reply/{messageId}` endpoint, where `{messageId}` is the `id` from the message, with a body `{"confirmed": true}` [74]. Example:

```
POST https://api.ibkr.com/v1/api/iserver/reply/07a13a5a-4a48-44a5-
bb25-5ab37b79186c
{ "confirmed": true }
```

If the order can now be accepted, the response will be a normal order acknowledgment (as shown earlier, with `order_id`, `order_status`, etc.) [75].

Also note the `messageIds` (like `"o163"`) in the reply. These indicate the category of the warning. You can choose to suppress these categories for the remainder of the session (see next section).

Order Reply Suppression

If you know you want to bypass certain precautionary warnings, you can suppress them so they don't interrupt order placement. The endpoint `/iserver/questions/suppress` allows this [76]. You `POST` an array of `messageIds` (like `"o163"`) to suppress [76]:

```
POST https://api.ibkr.com/v1/api/iserver/questions/suppress
{ "messageIds": [ "o163" ] }
```

The response confirms suppression: `{"status": "submitted"}` [77]. Once suppressed, a valid order ticket that would normally trigger that message will be accepted immediately (and an invalid one will be rejected) without the extra confirmation step [78] [79].

You can submit the suppression list at the **start of your session** (recommended) to avoid interruptions [80]. If suppressing additional message types mid-session, re-send the full list of `messageIds` you wish to suppress (the call does not accumulate them, it replaces the list) [81].

To remove all suppressions and restore all warnings, call `/iserver/questions/suppress/reset` (POST with no body) [82]. The response `{"status": "submitted"}` confirms that all message prompts are re-enabled [83].

Order Rejections

(Section placeholder – documentation coming soon.)

Previewing Orders

(Section placeholder – documentation coming soon.)

Modifying Orders

To modify an existing open (unfilled) order, use `POST /iserver/account/{accountId}/order/{orderId}` [84]. This submits a modification request for the specified order `{orderId}` in account `{accountId}`. Path parameters:

- `{accountId}` – The account containing the order.
- `{orderId}` – The order's ID (as returned when it was placed).

The body must be a single JSON object (***not*** an array) containing **all the original order parameters**, with only the intended changes made [85]. Essentially, to modify, you need to resend the entire order ticket as it currently stands, altering the fields you want to change [86]. All keys from the initial order must be present, and values unchanged except for those being modified [87].

> **Note:** Order modifications may be subject to different rules than initial submissions. You can use the `/iserver/secdef/rules` endpoint to check what rules apply for modifying a particular order (e.g. whether price can be increased/decreased by certain amounts) [88].

Example: Suppose we have an open buy limit order (orderId **987654**) in account DU123456 for 100 shares of AAPL at \$165 (the JSON body shown earlier) [89]. To raise the limit price to \$170, we call:

```
POST https://api.ibkr.com/v1/api/iserver/account/DU123456/order/987654
{
  "conid": 265598,
  "side": "BUY",
  "orderType": "LMT",
  "price": 170,
  "quantity": 100,
  "tif": "DAY"
}
```

Differences from the original: only `"price": 170` changed from 165 [90]. We must ensure all other attributes (conid, side, type, quantity, tif) remain exactly as before [91]. In the example, note:

- The request body is a single JSON object (not an array, since modifying one order) [92].
- We included all order parameters, only adjusting the price.

If successful, the response will look similar to a new order's, e.g.:

```
{
  "order_id": "987654",
  "order_status": "Submitted",
  "encrypt_message": "1"
}
```

It's also possible the modification triggers an *order reply message* (as described earlier) that needs confirmation [93].

Additionally, if you needed to recall the exact current parameters of an order (to modify it correctly), you can use the **Order Status** endpoint `/iserver/account/{accountId}/order/status/{orderId}` to fetch the order's details and then modify as needed [94].

Canceling Orders

To cancel an active order, use a `DELETE` request to `/iserver/account/{accountId}/order/{orderId}` [95]. Path parameters:

- `{accountId}` – Account containing the order.
- `{orderId}` – The order's ID to cancel.

No JSON body is needed for this DELETE [96]. Example:

```
DELETE https://api.ibkr.com/v1/api/iserver/account/DU123456/order/987654
```

A successful cancellation request returns a message indicating the request was received [97], for example:

```
{
  "msg": "Request was submitted",
  "order_id": 987654,
  "conid": 265598,
  "account": "DU123456"
}
```

Important: This response only confirms the cancellation request was submitted, not that the order was actually canceled. If an order is already executing or otherwise cannot be canceled (e.g. during an exchange auction), it might still fill despite the request [98]. Always verify order status after requesting a cancel.

Submitting Bracket Orders

(Section placeholder – documentation coming soon.)

Orders for Combos/Spreads

You can submit **combo (spread) orders** via the same endpoint `/iserver/account/{accountId}/orders`. For combos, instead of a single `conid`, you must use the `conidex` field to specify the combination of legs [99].

The `conidex` is a string encoding of the combo's composition:

```
{spread_conid};;;{leg_conid1}/{ratio},{leg_conid2}/{ratio},...
```

- `spread_conid`: a special ID representing the currency in which the combo trades. For US stock combos, this is the USD conid (as a plain integer). For combos in other currencies, it's `spread_conid@exchange` format [100]. See the table of available currency spread conids below.
- Followed by three semicolons `;;;`.
- Then each leg: `{leg_conid}/{ratio}`, with legs separated by commas [101].

In each leg, the **sign** of the ratio indicates buy or sell (positive = Buy, negative = Sell) and the magnitude is the relative size multiplier for that leg [102]. The combo order's price is calculated as the sum of each leg's price * ratio (taking into account sign) [103]. The number of legs allowed in a combo can vary by exchange [104].

Available currency spread `conid` values: (used as `spread_conid`) [105]

Currency	Spread ConID
AUD	61227077
CAD	61227082
CHF	61227087
CNH	136000441

Currency	Spread ConID
GBP	58666491
HKD	61227072
INR	136000444
JPY	61227069
KRW	136000424
MXN	136000449
SEK	136000429
SGD	426116555
USD	28812380

(Use the appropriate spread ConID for the combo's currency.)

Monitoring Live Orders

The endpoint `/iserver/account/orders` retrieves the status of all recently active orders for an account (both currently working and those filled/canceled in the same session) [106]. Example request:

```
GET https://api.ibkr.com/v1/api/iserver/account/orders?
filters=filled&force=true&accountId=U1234567
```

(The query parameters here, e.g. `filters=filled`, `force=true`, and `accountId`, can refine which orders to return and ensure an update from the server.)

A successful response includes an `"orders"` array with objects describing each order's status and details [107]. For example:

```
{
  "orders": [
    {
      "acct": "U1234567",
      "conidex": "265598",
      "conid": 265598,
      "account": "U1234567",
      "orderId": 1234568790,
      "cashCcy": "USD",
      "sizeAndFills": "5",
      "orderDesc": "Sold 5 Market, GTC",
      "description1": "AAPL",
      "ticker": "AAPL",
      "secType": "STK",
      "listingExchange": "NASDAQ.NMS",
      "remainingQuantity": 0.0,
```

```
      "filledQuantity": 5.0,
      "totalSize": 5.0,
      "companyName": "APPLE INC",
      "status": "Filled",
      "order_ccp_status": "Filled",
      "avgPrice": "192.26",
      "origOrderType": "MARKET",
      "supportsTaxOpt": "1",
      "lastExecutionTime": "231211180049",
      "orderType": "Market",
      "bgColor": "#FFFFFF",
      "fgColor": "#000000",
      "order_ref": "Order123",
      "timeInForce": "GTC",
      "lastExecutionTime_r": 1702317649000,
      "side": "SELL"
    }
  ],
  "snapshot": true
}
```


(Fields like `snapshot` indicate whether the data was retrieved from a cache or by forcing a refresh.)

Monitoring Executions

(Section placeholder – documentation coming soon.)

Portfolio and Positions

Querying Your Accounts

For non-tiered account structures, use `/portfolio/accounts` to get a list of account IDs under the user that can be queried for positions and account info [108]. This must be called before other `/portfolio` endpoints (which require an accountId from the list) [108]. Example:

```
GET https://api.ibkr.com/v1/api/portfolio/accounts
```

Response example [109]:


```
[
  {
    "id": "U1234567",
    "accountId": "U1234567",
    "accountVan": "U1234567",
    "accountTitle": "",
    "displayName": "U1234567",
    "accountAlias": null,
    "accountStatus": 1644814800000,
    "currency": "USD",
```

```
    "type": "DEMO",
    "tradingType": "PMRGN",
    "businessType": "IB_PROSERVE",
    "ibEntity": "IBLLC-US",
    "faclient": false,
    "clearingStatus": "O",
    "covestor": false,
    "noClientTrading": false,
    "trackVirtualFXPortfolio": true,
    "parent": {
      "accountId": "",
      "isMParent": false,
      "isMChild": false,
      "isMultiplex": false
    },
    "desc": "U1234567"
  }
]
```

For tiered structures (e.g. Financial Advisor or Broker with multiple subaccounts), `/portfolio/subaccounts` returns up to 100 subaccounts accessible [110]. If more than 100, use `/portfolio/subaccounts2` for the next set. (To list accounts the user can **trade**, use `/iserver/accounts` instead.)

Example:

```
GET https://api.ibkr.com/v1/api/portfolio/subaccounts
```

Response example (truncated) [111] [112]:

```
[
  {
    "id": "U1234567",
    "accountId": "U1234567",
    "accountVan": "U1234567",
    "accountTitle": "",
    "displayName": "U1234567",
    "accountAlias": null,
    "accountStatus": 1644814800000,
    "currency": "USD",
    "type": "DEMO",
    "tradingType": "PMRGN",
    "businessType": "IB_PROSERVE",
    "ibEntity": "IBLLC-US",
    "faclient": false,
    "clearingStatus": "O",
    "covestor": false,
    "noClientTrading": false,
    "trackVirtualFXPortfolio": true,
```

```
    "parent": {
      "accountId": "",
      "isMParent": false,
      "isMChild": false,
      "isMultiplex": false
    },
    "desc": "U1234567"
  }
]
```

Querying Currency Balances

The endpoint `/portfolio/{accountId}/ledger` provides cash balance information by currency for the specified account [113]. Example:

```
GET https://api.ibkr.com/v1/api/portfolio/{accountId}/ledger
```

Response example (showing USD and BASE currency entries) [114] [115]:

```
{
  "USD": {
    "commoditymarketvalue": 0.0,
    "futuremarketvalue": -1051.0,
    "settledcash": 214716688.0,
    "exchangerate": 1,
    "sessionid": 1,
    "cashbalance": 214716688.0,
    "netliquidationvalue": 215335840.0,
    "interest": 305569.94,
    "unrealizedpnl": 39695.82,
    "stockmarketvalue": 314123.88,
    "currency": "USD",
    "realizedpnl": 0.0,
    "funds": 0.0,
    "acctcode": "U1234567",
    "stockoptionmarketvalue": -2.88,
    "futuresonlypnl": -1051.0,
    "futureoptionmarketvalue": 0.0,
    "secondkey": "USD",
    "dividends": 0.0
  },
  "BASE": {
    "commoditymarketvalue": 0.0,
    "futuremarketvalue": -1051.0,
    "settledcash": 215100080.0,
    "exchangerate": 1,
    "sessionid": 1,
    "cashbalance": 215100080.0,
    "netliquidationvalue": 215721776.0,
```

```
    "interest": 305866.88,
    "unrealizedpnl": 39907.37,
    "stockmarketvalue": 316365.38,
    "currency": "BASE",
    "realizedpnl": 0.0,
    "funds": 0.0,
    "acctcode": "U1234567",
    "stockoptionmarketvalue": -2.88,
    "futuresonlypnl": -1051.0,
    "futureoptionmarketvalue": 0.0,
    "secondkey": "BASE",
    "dividends": 0.0
  }
}
```

(Above, `BASE` aggregates values in base currency.)

Querying Equity and Margin

Use `/portfolio/{accountId}/summary` for a wide range of account equity, margin, and balance values [116]. It returns many key-value pairs, both in aggregate for the whole account and by regulatory segment (if applicable) [117]. Example:

```
GET https://api.ibkr.com/v1/api/portfolio/{accountId}/summary
```

Excerpt of response (truncated for brevity) [118]:

```
{
  "accountcode": {
    "value": "U1234567",
    "timestamp": 1702582422000,
    "severity": 0
  },
  "indianstockhaircut": {
    "amount": 0.0,
    "currency": "USD",
    "isNone": false,
    "timestamp": 1702582422000,
    "value": null,
    "severity": 0
  },
  ...
}
```

(The response contains many fields; refer to documentation for full list.)

Advisor Features

(Section placeholder – documentation coming soon.)

FYIs, Alerts, and Bulletins

Types of Notification Messages

IBKR provides several types of notifications:

- **FYIs (For Your Information):** informational messages (e.g. about portfolio events or system changes). Some FYIs have associated disclaimers that users must acknowledge once.
- **Alerts:** user-defined alerts (e.g. price or volume triggers) that can notify via TWS pop-ups, email, etc., including Mobile Trading Assistant (MTA) alerts.
- **Bulletins:** system bulletins from IBKR (e.g. exchange status messages or other broadcast notifications).

(The documentation below covers FYI and Alert endpoints in detail.)

FYIs

Unread FYIs

To get the count of unread FYI notifications/disclaimers, call `GET /fyi/unreadnumber`. It returns an integer count [119]. For example, if there is 1 unread (and 3 total):

```
GET https://api.ibkr.com/v1/api/fyi/unreadnumber
```

Response:

```
{ "BN": 1 }
```

(In this context, `"BN"` *likely represents the count of unread bulletins/notifications.)*

All Recent FYIs

To retrieve a list of all recent notifications (FYIs), use `GET /fyi/notifications`. The response is a list of notifications, each indicating whether it's read (`"R": 0 or 1`), a timestamp, a title (`"MS"`), an FYI code (`"FC"`), a unique ID, and an HTML-formatted message (`"MD"`) [120] [121].

Example call:

```
GET https://api.ibkr.com/v1/api/fyi/notifications
```

Response (example snippet) [121] [122]:

```
[
  {
    "R": 0,
    "D": "1710847062.0",
    "MS": "FYI: Changes in Analyst Ratings",
    "MD":
"<html>Some investors use analysts ratings to stay informed...<br/> -
JWN@NYSE: ...</html>",
    "ID": "2024031947509444",
    "HT": 0,
    "FC": "PF"
  },
  {
    "R": 1,
    "D": "1710761985.0",
    "MS": "FYI: Changes in Analyst Ratings",
    "MD":
"<html>Some investors use analysts ratings to stay informed...<br/> -
F@NYSE: ...</html>",
    "ID": "2024031843829868",
    "HT": 0,
    "FC": "PF"
  },
  ...
]
```

Each object's `"MD"` contains an HTML message (which can include details like affected securities, etc.). The `"FC"` is the FYI code category.

Marking FYIs Read

To mark a specific FYI notification as read, use `PUT /fyi/notifications/{notificationID}`, where `{notificationID}` is the `"ID"` from the notification object [123]. (Multiple notifications under the same FYI code may exist; each can be marked read independently.)

Managing FYI Subscriptions

To retrieve the current subscription settings for FYI types, call `GET /fyi/settings`. The response is a list of objects, each with an FYI Code (`"FC"`), and flags indicating **Hidden** (`"H"`), **Active** (`"A"`), plus description fields [124]. For example [124]:

```
[
  {
    "FC": "PF",
    "H": 0,
    "A": 1,
    "FD": "Notify me of recent activity affecting my portfolio holdings.",
    "FN": "Portfolio FYIs"
  },
```

```
  {
    "FC": "PT",
    "H": 0,
    "A": 1,
    "FD": "Notify me of potential account configuration changes needed and useful features based on my position transfers.",
    "FN": "Position Transfer"
  }
]
```

Here, `"A": 1` means the user is subscribed (active) to that FYI category, `"H": 0` means not hidden.

To enable/disable a particular FYI subscription, use `POST /fyi/deliveryoptions/device` or related endpoints (not fully detailed in the excerpt). Each FYI code may also have an associated **disclaimer** that must be accepted once.

FYI Disclaimers

Some FYI categories require the user to accept a disclaimer before viewing detailed messages. Use `GET /fyi/disclaimer/{typecode}` to retrieve the disclaimer text for that FYI type (`typecode` = FYI code) [125]. For example, for code "PF":

```
{
  "FC": "PF",
  "DT": "This communication is provided for information purposes only and is not intended as a recommendation or a solicitation to buy..."
}
```

To acknowledge the disclaimer, call `PUT /fyi/disclaimer/{typecode}`. Once accepted, that disclaimer will not be shown again for that user for future notifications of that type [126].

FYI Delivery Options

(Section note: "TBD, needs to be revisited" – implying content to be added. Possibly refers to customizing delivery channels, like email or mobile push, for FYIs.)

Alerts

Alerts are user-created notifications that can be triggered by various conditions (price, time, margin, etc.) and deliver messages via TWS, mobile, or email. The Web API provides endpoints to create and manage alerts.

Creating and Modifying Alerts

Use `POST /iserver/account/{accountId}/alert` to **create a new alert or modify an existing alert** [127]. This endpoint requires a JSON body specifying the alert details. **Path parameter:** `{accountId}` is the account for which the alert is set.

Required Body Parameters: (for creation) [128] [129]

- `alertName` (String, required) – A human-readable name for the alert.
- `alertMessage` (String, required) – The message content of the alert (what it reports when triggered).
- `alertRepeatable` (int, required) – 0 or 1 indicating if the alert can trigger more than once. For Mobile Trading Assistant (MTA) alerts, use 1.
- `email` (String, required if `sendMessage` == 1) – Email address to send the alert to (if email notifications are enabled).
- `expireTime` (String, required if `tif` == "GTD") – When using "GTD" (Good-Till-Date) for time-in-force, this is the expiry datetime for the alert in format `"YYYYMMDD-HH:mm:ss"`.
- `iTWSOrdersOnly` (int, optional) – 0/1 flag; if 1, triggers only generate mobile app notifications (no email/TWS pop-ups).
- `outsideRth` (int, required) – 0/1 flag; if 1, allow the alert to trigger outside regular trading hours.
- `sendMessage` (int, optional) – 0/1; if 1, enable sending email for the alert.
- `showPopup` (int, optional) – 0/1; if 1, enable TWS pop-up notification for the alert.
- `tif` (String, required) – Time in force for the alert's monitoring: either "GTC" (good-till-cancelled) or "GTD" (good-till-date).
- `conditions` (List of Objects, required) – A list of one or more condition objects that define when the alert triggers [130].
- Each condition object includes:
  - `conidex` (String, required) – the instrument identifier in `conid@exchange` format that the condition applies to [131].
  - `logicBind` (String, required) – logic for multiple conditions: "a" (AND), "o" (OR), "n" (END – for a single condition or the last condition in a series) [132].
  - `operator` (String, required) – comparison operator, e.g. ">=", "<=", etc., indicating how to compare the current value to the threshold [133].
  - `timeZone` (String, required for time-based conditions) – relevant timezone, if applicable (e.g. for time-triggered alerts) [134].
  - `triggerMethod` (String or int, required) – usually "0"; reserved field for specifying the price type trigger method (default 0).
  - `type` (int, required) – the condition type code (1=Price, 3=Time, 4=Margin, 5=Trade, 6=Volume, 7=MTA Market, 8=MTA Position, 9=MTA Account Daily PnL, etc.) [135].
  - `value` (String, required) – the trigger value (e.g. price level or time). Can be `"*"` for certain conditions as a wildcard. For time conditions, use format `"YYYYMMDD-HH:mm:ss"` [136].

For example, a JSON body to create an alert for AAPL price dropping below 183.34 might look like [137] [138]:

```
{
  "alertMessage": "AAPL Price Drop!",
  "alertName": "AAPL_Price",
  "expireTime": "20270101-12:00:00",
  "alertRepeatable": 0,
  "outsideRth": 0,
  "sendMessage": 1,
  "email": "user@domain.net",
  "iTWSOrdersOnly": 0,
```

```
  "showPopup": 0,
  "tif": "GTD",
  "conditions": [{
    "conidex": "265598@SMART",
    "logicBind": "n",
    "operator": "<=",
    "triggerMethod": 0,
    "type": 1,
    "value": "183.34"
  }]
}
```

The endpoint returns a **Response Object** with fields including [139]:

- `request_id` (integer, often null for alert requests),
- `order_id` (integer, the alert's ID),
- `success` (boolean),
- `text` (message, e.g. "Submitted" if successful),
- `order_status` (string, often null for alerts),
- `warning_message` (string, usually null).

Example response for a successful creation [140]:

```
{
  "request_id": null,
  "order_id": 9876543210,
  "success": true,
  "text": "Submitted",
  "order_status": null,
  "warning_message": null
}
```

If modifying an existing alert, the same endpoint is used but the provided `alertName`, `alertMessage`, etc. will update the alert with a new `order_id`. (When an alert is modified, IBKR treats it as creating a new alert instance with a new ID.)

Listing Configured Alerts

To get all alerts for a given account, use `GET /iserver/account/{accountId}/alerts` [141]. This returns an array of alert summaries. **Path param:** the account ID.

Example:

```
GET /iserver/account/U1234567/alerts
```

Response (array of objects) [142] [143]:

```
[
  {
    "order_id": 9876543210,
    "account": "U1234567",
    "alert_name": "AAPL Price",
    "alert_active": 1,
    "order_time": "20231211-18:55:35",
    "alert_triggered": false,
    "alert_repeatable": 0
  }
]
```

Each alert in the list includes its `order_id` (alert ID), `account`, name, whether it's active, creation time, whether it's triggered, and if it's repeatable [143].

Retrieving Alert Details

To get detailed information on a specific alert, use `GET /iserver/account/alert/{order_id}?type=Q` [144]. The `order_id` is the alert's ID, and a query parameter `type=Q` must be provided (as per documentation, always use `"Q"` for this call) [145]. This returns an object with all the alert's settings.

Path param: `{order_id}` – the alert's ID.
Query param: `type=Q` (literally the letter Q).

Example:

```
GET /iserver/account/alert/9876543210?type=Q
```

The **Response Object** includes numerous fields describing the alert [146] [147] [148], such as:

- `account` – account ID.
- `order_id` – alert ID.
- `alert_name` – name of the alert.
- `tif` – time in force ("GTC" or "GTD").
- `expire_time` – if GTD, the expiration date/time.
- `alert_active` – 1 or 0 if the alert is currently active/enabled.
- `alert_repeatable` – 1 or 0 if it can trigger multiple times.
- `alert_email` – the email address for notifications (if any).
- `alert_send_message` – 1/0 for whether email is enabled.
- `alert_message` – the text of the alert notification.
- `alert_show_popup` – 1/0 for TWS pop-up.
- `alert_play_audio` – 1/0 for sound alert.
- `order_status` – typically "Presubmitted" for active alerts.
- `alert_triggered` – whether the alert has triggered yet (true/false).
- `fg_color`, `bg_color` – UI colors (can usually be ignored).
- `order_not_editable` – bool, whether the alert (order) can be edited.
- `itws_orders_only` – 1/0, if it triggers mobile notifications only.
- `alert_mta_currency` – for MTA alerts, the currency code (e.g. "USD").

- `alert_mta_defaults` – MTA default values.
- `tool_id` – for MTA alerts, a tracking ID; null for regular alerts.
- `time_zone` – for time-based alerts, the time zone used.
- `alert_default_type` – default alert type (if configured).
- `condition_size` – number of conditions in the alert.
- `condition_outside_rth` – 1/0 if allowed outside RTH.
- `conditions` – array of condition objects (each with fields like `condition_type`, `conidex`, `contract_description_1`, `condition_operator`, `condition_trigger_method`, `condition_value`, `condition_logic_bind`, `condition_time_zone`) describing each condition [149] [150] [151].

Example partial detail (continuing from the previous list example) [152] [153] [154]:

```
{
  "account": "U1234567",
  "order_id": 9876543210,
  "alert_name": "AAPL Price",
  "tif": "GTD",
  "expire_time": "20231231-12:00:00",
  "alert_active": 1,
  "alert_repeatable": 0,
  "alert_email": null,
  "alert_send_message": 0,
  "alert_message": "MTA TEST!",
  "alert_show_popup": 0,
  "alert_play_audio": null,
  "order_status": "Submitted",
  "alert_triggered": false,
  "fg_color": "#FFFFFF",
  "bg_color": "#0000CC",
  "order_not_editable": false,
  "itws_orders_only": 0,
  "alert_mta_currency": null,
  "alert_mta_defaults": null,
  "tool_id": null,
  "time_zone": null,
  "alert_default_type": null,
  "condition_size": 1,
  "condition_outside_rth": 0,
  "conditions": [
    {
      "condition_type": 1,
      "conidex": "265598@SMART",
      "contract_description_1": "AAPL",
      "condition_operator": "<=",
      "condition_trigger_method": "0",
      "condition_value": "183.34",
      "condition_logic_bind": "n",
      "condition_time_zone": null
    }
```

```
  ]
}
```

Mobile Trading Alerts (MTA)

MTA alerts are a special built-in alert (one per user) for mobile notifications. You cannot create/delete MTA alerts; one exists by default. The endpoints treat it slightly differently: to get the MTA alert info, use `GET /iserver/account/mta` [155] [156]. This returns the same fields as a normal alert detail (as above). Modifying the MTA alert is done via the regular alert endpoint (as a modification). If you "delete" the MTA alert via the delete endpoint, it resets to default settings (does not permanently remove, since it always exists) [157] [158].

Example of retrieving MTA alert:

```
GET /iserver/account/mta
```

Response example (showing default/inactive state) [159] [160]:

```
{
  "account": "U1234567",
  "order_id": 9998887776,
  "alert_name": null,
  "tif": "GTC",
  "expire_time": null,
  "alert_active": 1,
  "alert_repeatable": 1,
  "alert_email": null,
  "alert_send_message": 1,
  "alert_message": null,
  "alert_show_popup": 0,
  "alert_play_audio": null,
  "order_status": "Inactive",
  "alert_triggered": false,
  "fg_color": "#000000",
  "bg_color": "#AFAFAF",
  "order_not_editable": false,
  "itws_orders_only": 0,
  "alert_mta_currency": "USD",
  "alert_mta_defaults":
"9:STATE=1,MIN=-43115000,MAX=43115000,STEP=500,DEF_MIN=-4311500,DEF_MAX=4311500|...",
  "tool_id": 55834574848,
  "time_zone": "GMT (GMT),GMT (Africa/Abidjan),...",
  "alert_default_type": null,
  "condition_size": 0,
  "condition_outside_rth": 0,
  "conditions": []
}
```

(Above: `order_status` "Inactive" indicates the MTA alert is not currently customized/active beyond defaults. The `alert_mta_defaults` and `tool_id` fields are specific to MTA.)

Activating/Deactivating Alerts

To **activate or deactivate** an existing alert, use `POST /iserver/account/{accountId}/alert/activate` [161]. You must include a JSON body with the alert's ID and the desired active state. **Path param:** accountId. **Body params:**

- `alertId` (int, required) – the alert's order_id.
- `alertActive` (int, required) – 1 to activate (enable notifications), 0 to deactivate (silence it).

This call does not delete the alert; it just toggles its active status (an inactive alert remains defined but will not trigger until reactivated) [157].

Example body to deactivate an alert ID 9876543210:

```
{
  "alertId": 9876543210,
  "alertActive": 0
}
```

Response object includes [162] [163]:

- `request_id` (int, usually null),
- `order_id` (int, the alertId that was acted on),
- `success` (bool),
- `text` (string message),
- `failure_list` (list of any alerts that failed to toggle, if any).

Example success response [163]:

```
{
  "request_id": null,
  "order_id": 9876543210,
  "success": true,
  "text": "Request was submitted",
  "failure_list": null
}
```

Deleting Alerts

To **permanently delete** an alert, use `DELETE /iserver/account/{accountId}/alert/{alertId}` [164]. Path params: accountId, and alertId (the order_id of the alert). Deleting the MTA alert (alertId for MTA) will **reset** it to default state (since MTA alert cannot be truly removed) [165]. If you pass alertId = 0, **all** alerts for that account will be deleted [158].

Example:

```
DELETE /iserver/account/DU123456/alert/9876543210
```

No body required (though sending an empty JSON `{}` is fine).

Response object is similar to the activate response, indicating success or failure for the given alertId(s) [166] [167]. Example:

```
{
  "request_id": null,
  "order_id": 9876543210,
  "success": true,
  "text": "Request was submitted",
  "failure_list": null
}
```

Bulletins

Bulletins are system-wide or exchange messages broadcast to clients. (There are no specific interactive endpoints for bulletins in this documentation excerpt aside from retrieving unread counts via FYI endpoints.) Bulletins appear among FYI notifications and can be counted via `/fyi/unreadnumber` as shown above.

(End of Part I documentation content.)

Part II: IBKR Web API Reference (Beta)

IBKR's unified Web API Reference is currently in beta and subject to change [168]. Not all endpoints of the Web API are yet included in the Swagger-based reference interface. In the meantime, IBKR directs users to the existing separate documentation for the constituent APIs:

- **Trading Web API (Client Portal API)** – covers trading-related endpoints.
- **Account Management Web API** – covers account management (account setup, funds, etc.).
- **Flex Web Service** – covers Flex report request/retrieval API.

These legacy docs remain available at the IBKR website [169]. Once the unified reference is complete, it will provide per-endpoint details in one place.

(The reference interface would list endpoints and their parameters in an interactive fashion; users are advised to refer to it for the most up-to-date endpoint definitions. As of this writing, not all endpoints are present in the reference, so using the comprehensive documentation below (Client Portal API v1.0 Documentation) may be necessary.)

Legacy API Documentation Links:

- [Trading Web API] – Client Portal Web API (trading endpoints) documentation [169].
- [Account Management Web API] – Account management endpoints documentation [169].
- [Flex Web Service] – Flex report API documentation [169].

(These links refer to Interactive Brokers' website sections for each API.)

Part III: Client Portal Web API v1.0 Documentation (Detailed Reference)

(This section is the comprehensive documentation of the Client Portal Web API version 1.0, which is the core of IBKR's Web API. It includes detailed requirements, all REST endpoints, and WebSocket topics. Code examples are given in Python, cURL, and sometimes other languages. The content below is extracted and summarized from IBKR's IBKR Campus documentation page [170] [171].)

Introduction

Interactive Brokers' **Client Portal Web API** provides real-time access to IBKR's trading system – including features like live market data, market scanners, and intra-day portfolio updates – via both synchronous REST calls and asynchronous WebSocket feeds [172]. Multiple authorization methods are supported (OAuth 1.0a, OAuth 2.0, SSO, and the Client Portal Gateway tool) to suit different use cases [173]. An active **IBKR account (IBKR Pro)** is required, and must be fully opened and funded to use the API [174]. If you don't have an account, you can create one (wait until it's activated). Note that the Client Portal API is not available for IBKR Lite accounts [175].

Requirements & Limitations

To use the Web API (Client Portal API), the following are required [176]:

1. An open, active IBKR account (paper trading only works in association with a funded live account; demo accounts cannot get live data) [177].
2. IBKR Pro account type.
3. The account must be funded (non-zero balance).

Canadian Residents Restricted from API Trading Canadian Products

Important for Canadian clients: Interactive Brokers Canada (IBC) **does not permit** using the API (or any third-party tool) to electronically submit orders for products on Canadian exchanges [178]. This restriction comes from CIRO (formerly IIROC) Rule 3200, which prohibits automated order systems for client orders in Canada [179]. In practice, this means if you are a client of IB Canada, you cannot use the API to trade Canadian-listed securities (including via third-party platforms like TradingView or NinjaTrader) [180]. Orders for other markets via API are allowed, but Canadian-listed instruments must be traded manually (through TWS or Client Portal).

WebAPI Basics Tutorial

IBKR provides a tutorial series covering common Web API tasks, with examples using Python (the concepts apply similarly in other languages) [181]. The tutorial includes:

- Running the Client Portal Gateway
- Requesting live and historical market data
- Placing and monitoring orders
- Reviewing account information (including Financial Advisor account structures)
- Using market scanners

- Implementing a WebSocket connection

The tutorial can be found on IBKR's website [182] (IBKR Client Portal API Tutorial). It's recommended for new API users to get familiar with workflow.

Client Portal Gateway

For individual (retail) accounts, IBKR requires the use of the **Client Portal Gateway** to authenticate to the Web API [183]. The Client Portal Gateway is a lightweight Java application that runs locally and facilitates the login process (including two-factor authentication) and routes API requests to IB's backend.

Limitations of the Client Portal Gateway

Most Web API endpoints work via both the Gateway and direct OAuth, but note these limitations for the Gateway mode [184]:

- You must log in via a web browser on the **same machine** where the Gateway is running (to authenticate the session) [185].
- All API calls must originate from the **same machine** where the Gateway was authenticated (i.e., you generally send requests to `https://localhost:5000` on that machine) [186].
- Endpoints under `/gw/api`, `/oauth`, or `/oauth2` are **not supported** via the Gateway (those are used for direct OAuth flows and are not needed when using the Gateway) [187].

How to Download and Run the Gateway

Prerequisite – Java: The Client Portal Gateway is a Java application. You need a Java Runtime Environment (JRE) installed (minimum Java 8 update 192) [188]. Check if Java is installed by running `java -version` in a terminal [189]. If not installed or version is too low, download it from the official site [190].

Download Gateway: Download the Client Portal API Gateway (either Standard or Beta release). IBKR offers both; if the standard release has issues, try the Beta [191]. Downloads are available on IB's website [192]. Unzip the package to a convenient location.

Run Gateway: To start the Gateway, you must run it via command line (it cannot be launched by double-clicking, as it requires arguments) [193]. By default, the Gateway listens on port 5000 for API requests [194]. If needed, you can change the port by editing the `root/conf.yaml` file (see below).

Steps to run: open a Terminal/Command Prompt, navigate (`cd`) to the gateway's directory (e.g., `clientportal.gw` folder) [195]. Then execute:

- On Windows: `bin\run.bat root\conf.yaml` [196]
- On macOS/Linux: `bin/run.sh root/conf.yaml` [197]

This starts the gateway. Keep this terminal open and running while using the API.

Once running, proceed to authenticate via your browser: open `https://localhost:5000` in a web browser on that machine. You should see a login page; enter your IBKR credentials [198] and complete any two-factor authentication. On success, you will see "Client login succeeds" in the browser [198], and the gateway terminal will indicate an active session.

For **paper trading**, note: you must use your paper account's **username** (which is different from your live account username). You can find the paper username in Client Portal under Settings > Paper Trading Account [199]. Use that paper username and its password to log in if you want an isolated paper session.

How To Modify the Client Portal Gateway Port

If port 5000 is in use or you prefer a different port, you can change it in the configuration [200]: open the `conf.yaml` in the gateway's `root` directory, and edit the `listenPort` value (default 5000) to another number (e.g., 5001) [201]. Save the file and restart the gateway. Example `conf.yaml` snippet (changing port to 5001) [202]:

```
ip2loc: "US"
proxyRemoteSsl: true
proxyRemoteHost: "https://api.ibkr.com"
listenPort: 5001
listenSsl: true
...
```

After modifying, launch the gateway and use `https://localhost:5001` for API calls.

Client Portal Gateway FAQ

Common issues and questions:

- **Browser warning about insecure connection / invalid SSL certificate?**
  Why: The gateway uses a self-signed certificate by default. Browsers will warn that the connection to `localhost` is not trusted [203].
  Solution: It's expected behavior; the connection between your browser and your local gateway is "insecure" only in that context (it's within your machine). All traffic from the gateway to IBKR is still secure (SSL) [204]. You can proceed past the warning or install a certificate if desired.

- **Can I automate the Gateway login process?**
  Currently no supported mechanism to fully automate the interactive login (entering username/password and 2FA) for individual accounts [205]. Third-party hacks exist but IBKR does not endorse or support them (they may pose security risks) [206]. Each day (after midnight) you'll need to log in once via the browser to establish a session [207].

- **How often do I need to log in via the browser?**
  At least once a day (after the nightly server reset) [207]. After midnight (server time), the session will require reauthentication. In practice, you log in once a day per account.

- **What if I get "Server listen failed: Address already in use" error on Mac?**
  This means something (often Mac's system AirPlay or other service) is already using port 5000 on localhost [208]. Solution: change the Gateway port as described above to an unused port (and restart it) [209].

Once logged in, your gateway remains active until you log out or the session expires (e.g., after maintenance). You can monitor the gateway's console for messages.

Authentication

After launching the Client Portal Gateway and logging in via the browser, you have an active session on `localhost:5000` (or your chosen port) [198]. From this point, you can use the API. **If using direct OAuth flows (for institutions or third-party apps):** you would obtain an OAuth token via the documented OAuth endpoints, but for individual retail via Gateway, the login process we just did suffices to authenticate.

Session Authentication

The Web API requires a two-tier session as described earlier (read-only + brokerage). For **individual (Gateway) users**, logging into the gateway creates the read-only session and automatically also initializes the brokerage session (since the gateway is designed to do that) [210] [211]. Thus, once login succeeds, you typically have full access (both tiers active) unless your account was already trading elsewhere (in which case the gateway might only have read-only until you explicitly start trading via an endpoint).

Key points about sessions (recap) [212] [213]:

1. **One trading session per username.** If you log into TWS and then try to trade via API, one of the sessions will drop. The gateway prevents that by initially only establishing read-only if it detects an active session [214] [215].
2. **Read-only vs brokerage session:** The outer session allows non-`/iserver` calls (account info, etc.) without disturbing an existing trading session [211]. The inner session (brokerage) is needed for `/iserver` calls (trades, live data) [215].
3. The Client Portal Gateway will **automatically initialize** the brokerage session upon login by default (since its purpose is to enable trading) [216].
4. If using OAuth (without gateway) and your username wasn't in another session, you'd log in and then call `/iserver/auth/ssodh/init` to start a brokerage session manually [217]. The Gateway does this for you, so you usually don't need to call it explicitly in that scenario.
5. The brokerage session applies to all accounts under the username (once authenticated, you can select any of your accounts for trading) [218]. You don't need separate sessions per account.
6. Endpoints not under `/iserver` (like `/portfolio` endpoints) do **not** require an active brokerage session, only the read-only session [219].

Background regarding CP Web API sessions

(The above key points cover much of this background. The documentation presented a numbered list reiterating the two-tier nature and the difference between Portal (read-only) and brokerage sessions, which we have summarized.) [220] [221] [222]

Understanding Brokerage Sessions

A brokerage (trading) session is tied to a username and grants access to all accounts that username can trade [223]. Only one such session can be active per username. If you have multiple users (logins) under a master, each is separate. The session carries the user's trading permissions and market data subscriptions. This concept was explained earlier in Part I and above.

Understanding Interactive Brokers terminology

(This section clarifies terms: essentially reiterating that a "brokerage session" = a trading session associated with a username; accounts are pools of equity accessible by that username; trading permissions are at the username level, etc.) [223]

Managing Multiple Sessions

If you want to use multiple IBKR platforms simultaneously (say TWS and the Web API) without logging one out, the only way is to have multiple usernames for the same account structure [224]. IBKR allows creation of secondary usernames (with certain permissions) for an account or account family. For example, a financial advisor could have one login for TWS and another for API. If you try to use the same username, one session will log out the other due to the one-session rule [225].

IBKR Knowledge Base has info on requesting an additional username for an account if needed [226]. Note also that market data subscriptions are per username, so an additional username may require its own subscriptions (fees) if it's used for data [227].

Using a Paper Account

It's strongly recommended to test API calls in the Paper Trading system before live trading [228]. To use the paper account via API, you must log in with the **paper trading username** (not the live username). Steps to find or set your paper login credentials [229]:

1. Log in to Client Portal with your main (live) username.
2. Go to the user menu (head & shoulders icon) > Settings.
3. Under Account Settings, find **Paper Trading Account**.
4. Note the Paper Trading **Username** and **Account Number**. You can also set a password for the paper login if needed.

Then use that paper username and password when authenticating (via Gateway or OAuth). Once in paper mode, all actions affect the paper account only.

Authentication Frequently Asked Questions

Common questions about session status and timeouts:

- **How can I tell if my brokerage session is authenticated?**
  Use `GET /iserver/auth/status`. It returns a JSON indicating whether you are authenticated (and sometimes whether currently connected). If `"authenticated": true`, you have a valid brokerage session [230]. You should call this after login to verify your session, and if needed, call `/iserver/auth/ssodh/init` to establish a brokerage session if it's not authenticated (e.g., if `authenticated` is false but `connected` is true, meaning you have a connection without an active trading session) [231] [232].

- **How long does a session remain authenticated?**
  Up to 24 hours (resets at the next regional server reset – which occurs at midnight US Eastern Time for Americas, midnight CET for Europe, midnight HKT for Asia) [233]. However, inactivity will time out the session much sooner: about 5-6 minutes of no requests will cause a session to require re-auth (though the connection might remain) [234]. In practice, IB recommends sending

periodic keep-alive calls (see next question) and re-log in after the standard daily maintenance anyway [231].

- **How can I prevent the session from timing out?**
  Call the `/tickle` endpoint periodically (recommended once per minute) [235]. This endpoint is a lightweight no-op that refreshes the session timer. If the session does time out (after ~5 min of inactivity), `/auth/status` will show `authenticated:false` (but `connected:true` if the gateway still has a link). In that case, calling `/iserver/auth/ssodh/init` will reinitialize the brokerage session without needing a full re-login [232]. Essentially, `/tickle` is your keep-alive ping.

- **Is it possible to authenticate a live session without 2FA device?**
  No, two-factor authentication is **mandatory** for IBKR account login (unless the account is configured with an alternate 2FA method). The login process for the API gateway is the same as Client Portal – you must perform 2FA if prompted [236]. IBKR requires 2FA for security whenever accessing sensitive info or trading functionality. There is no API bypass for 2FA.

Pacing Limitations

Interactive Brokers imposes certain **pacing limits** on API requests [237] (to protect system performance):

- **Global limit:** 10 requests per second for all endpoints combined (per user session) [238].
- **Per-endpoint limits:** Some endpoints have specific lower limits, detailed in a table (see below) [239]. If an endpoint is not listed, assume the global 10/sec limit applies [240].

If you exceed a limit, you get HTTP 429 Too Many Requests. The offending IP will be blocked for 15 minutes ("penalty box") [241]. After 15 minutes, it's removed until another violation. Repeat violators can be blocked indefinitely until the issue is resolved [241].

Endpoint Rate Limit Table: (similar to Part I, but note the global limit is 10/sec here, likely reflecting older info. We list the known ones) [242] [243]:

- /fyi/unreadnumber – 1 req/sec
- /fyi/settings – 1 req/sec
- /fyi/settings/{typecode} – 1 req/sec
- /fyi/disclaimer/{typecode} – 1 req/sec
- /fyi/disclaimer/{typecode} (PUT) – 1 req/sec
- /fyi/deliveryoptions – 1 req/sec
- /fyi/deliveryoptions/email – 1 req/sec
- /fyi/deliveryoptions/device – 1 req/sec
- /fyi/deliveryoptions/{deviceId} – 1 req/sec
- /fyi/notifications – 1 req/sec
- /fyi/notifications/more – 1 req/sec
- /fyi/notifications/{notificationId} – 1 req/sec
- /iserver/account/orders – 1 req/5 sec
- /iserver/account/pnl/partitioned – 1 req/5 sec
- /iserver/account/trades – 1 req/5 sec
- /iserver/marketdata/history – max 5 concurrent requests
- /iserver/marketdata/snapshot – 10 req/sec
- /iserver/scanner/params – 1 req/15 min
- /iserver/scanner/run – 1 req/sec

- /pa/performance – 1 req/15 min
- /pa/summary – 1 req/15 min
- /pa/transactions – 1 req/15 min
- /portfolio/accounts – 1 req/5 sec
- /portfolio/subaccounts – 1 req/5 sec
- /sso/validate – 1 req/min
- /tickle – 1 req/sec

(These limits match those in Part I, except the global 50/sec vs 10/sec difference. It's possible IBKR increased the global limit from 10 to 50 in newer versions. Always check latest docs.)

Regular Server Maintenance

IBKR performs daily server resets for the trading system, typically at midnight server time for each region [244]. During the reset, active sessions will disconnect. The times are:

- North America: 00:00 Eastern Time (which is 20:00 UTC) [245].
- Europe: 00:00 CET (23:00 UTC or 22:00 UTC depending on daylight savings) [245].
- Asia: 00:00 HKT (16:00 UTC) [245].

Plan to reconnect after these times. It's good practice to program your app to logout before this time and reconnect after, or handle the disconnect gracefully.

Endpoints (REST API)

Base URL: When using the Client Portal Gateway, the base URL for API calls is `https://localhost:5000/v1/api` [246]. (If not using the gateway and instead using direct network access with OAuth token, the base would be `https://api.ibkr.com/v1/api` [247].) The examples below use a variable `baseUrl` to represent the appropriate prefix.

SSL Note: The Gateway's default certificate is self-signed, so your HTTP client might need to disable certificate verification or provide the certificate. In cURL, add `--insecure`; in Python `requests`, use `verify=False` [248] [249]. This will avoid SSL errors when connecting to localhost. (Alternatively, one can install a valid certificate for the gateway.)

Headers: All requests should include the following HTTP headers [250]:

- `Host: api.ibkr.com` (even though you connect to localhost, set Host to IBKR's API host) [251].
- `User-Agent: ...` (not strictly required, but best practice to include one identifying your application or use a common browser agent string) [252].
- `Accept: */*` (accept any response type, JSON is typically returned) [253].
- `Connection: keep-alive` (persistent connection) [254].
- `Content-Length: ...` (for POST/PUT with a body, ensure this header is included; most HTTP libraries handle it automatically, but if constructing raw requests, include it or IB will return 411 Length Required) [255].

Now we list major endpoint categories and representative endpoints with their usage. Due to the breadth of the API, not every endpoint is exhaustively described here, but major categories are covered with at least one example each. (For full endpoint details, refer to IBKR's official documentation or the sections above in Part I which overlap with many endpoints.)

Alerts Endpoints (REST)

- **Create/Modify Alert** – `POST /iserver/account/{accountId}/alert` – *Create a new or update an existing alert.* Requires JSON body with alert details (see **Alerts** section above for fields). Returns alert `order_id` and status [127] [140].
- **List Alerts** – `GET /iserver/account/{accountId}/alerts` – *Get all alerts for the account.* Returns an array of alert summary objects [141] [143].
- **Alert Details** – `GET /iserver/account/alert/{order_id}?type=Q` – *Get detailed info for a specific alert.* Returns full alert object (conditions, etc.) [144] [147].
- **MTA Alert** – `GET /iserver/account/mta` – *Get Mobile Trading Assistant alert info.* (Similar output to alert details, for the special MTA alert) [155] [159].
- **Activate/Deactivate Alert** – `POST /iserver/account/{accountId}/alert/activate` – *Toggle an alert's active status.* JSON body: `{"alertId": <id>, "alertActive": 0|1}` [157] [256]. Returns success/failure info [163].
- **Delete Alert** – `DELETE /iserver/account/{accountId}/alert/{alertId}` – *Delete a specific alert (or all alerts if alertId=0).* Returns success info [164] [167].

Accounts Endpoints

- **Account Profit and Loss** – `GET /iserver/account/pnl/partitioned` – Returns an object with P&L figures (daily P&L, unrealized P&L, net liquidity, etc.) for the account and its sub-accounts or models [257] [258]. No parameters. Example response given above [259].
- **Search Dynamic Account** – `GET /iserver/account/search/{searchPattern}` – *For brokers with dynamic account numbers (DYNACCT property), search for accounts by a pattern.* Only applicable to IBroker accounts (not individuals) [260] [261]. Query param is embedded in path as `{searchPattern}` (e.g., first few characters of an account ID). Returns a list of matching accounts with fields: `accountId`, `alias`, `allocationId` [262] [263].
- **Set Dynamic Account** – `POST /iserver/dynaccount` – *For DYNACCT broker users, set the active account for subsequent requests.* Body: `{"acctId": "ACCOUNT_ID"}` to specify which account future calls should act on [264] [265]. After this, requests will pertain to that account until changed. Returns confirmation: `{"set": true, "acctId": "..."}` [266].
- **Signatures and Owners** – `GET /iserver/account/signature` and related endpoints – *Retrieve digital envelope signature and account owner info.* (These endpoints allow verification of account ownership and retrieving digital signature for certain requests – detailed documentation not excerpted above, but mentioned in nav.)
- **Switch Account** – `POST /iserver/accounts` – *Switch the currently active account for a user with multiple accounts.* (This might be relevant for multi-account users, likely providing an account ID in the body to switch the default context. Not fully detailed above.)
- **Receive Brokerage Accounts** – `GET /iserver/accounts` – *Get a list of accounts the user can trade.* For multi-account users, returns accounts under the brokerage session. Essentially similar to `/portfolio/accounts` but for trading context.

(Note: The documentation above did not explicitly show the endpoints "Signatures and Owners" or "Switch Account" in the text we extracted, but they are listed in the nav. These likely correspond to endpoints for multi-account management, such as confirming which account is active for trading if multiple are available. In practice, `/iserver/accounts` returns a list of tradable accounts and maybe sets a default, and `/iserver/account/pnl/partitioned` etc. require an active account. The dynamic account endpoints are specifically for IBrokers using DYNACCT.)

Contract and Market Data Endpoints

The **Contract** category includes many endpoints for retrieving contract details and trading schedules:

- **Search by Conid** – e.g. `GET /iserver/contract/{conid}/info` (if exists) to get contract details for a given conid.
- **All Conids by Exchange** – possibly an endpoint to list all contracts on an exchange.
- **SecDef (Security Definition) Search** – `POST /trsv/secdef` – search contracts by underlying symbol and filters.
- **Contract Rules** – `GET /iserver/secdef/rules` – get market rule details for order placement (e.g., price increment rules).
- **Futures/Options Strikes** – `GET /iserver/contract/{underlyingConid}/secdefstrikes` to get available strikes for options.
- **Algo Params** – search algorithm parameters by conid (for IB Algos).
- **Bond Info** – search bond details by filter.
- **Trading Schedule** – endpoints to get trading hours for a given contract or market.

(These were listed in the nav: e.g., "Search Contract by Symbol", "Trading Schedule (NEW)", etc., indicating endpoints for retrieving contract details and trading hours. The specifics would be documented under each item.)

Market Data endpoints (beyond what we covered in Part I):

- **Market Data Snapshot** – `GET /iserver/marketdata/snapshot` (covered above in detail).
- **Market Data Streaming** – handled via WebSocket messages (see WebSockets section) for top-of-book and deeper data.
- **Historical Data** – `GET /iserver/marketdata/history` – retrieve historical prices/bars for a contract (with parameters for timeframe, bar size, etc.). Subject to a concurrency limit (5 concurrent requests) [267]. If more than 5 history requests are made at once, you must queue them.
- **Market Data Fields** – an enumeration of field codes (provided in documentation reference).
- **Unavailable Historical Data** – certain data might not be available (the doc likely lists cases when data can't be retrieved).
- **Regulatory Snapshot** – perhaps an endpoint to get NBBO snapshot if required by regulations (often IB provides a separate endpoint for US equities NBBO snapshot if you don't have subscriptions).
- **Unsubscribe** – as listed: there might be an endpoint to unsubscribe a specific market data stream or all streams (the nav shows "Unsubscribe (Single)" and "Unsubscribe (All)"). Possibly these are WebSocket messages or REST calls to cancel market data subscriptions.

Order Monitoring Endpoints

- **Live Orders** – `GET /iserver/account/orders` – retrieve list of live (and recent) orders (covered above in *Monitoring Live Orders*). Optionally can filter by status.
- **Order Status** – `GET /iserver/account/{accountId}/order/status/{orderId}` – get detailed status of a specific order (e.g., filled quantity, remaining, etc.).
- **Trades** – `GET /iserver/account/trades` – get a list of executed trades (fills) for the account (maybe within current session or day). Possibly with filtering.

Orders Endpoints (Placement and Management)

We covered many in Part I. To summarize and add:

- **Place Order** – `POST /iserver/account/{accountId}/orders` – place new order(s). Returns orderId or reply message (if confirmation needed).
- **Place Order Reply** – `POST /iserver/reply/{replyid}` – respond to an order reply (with `"confirmed": true/false`).
- **WhatIf/Preview Order** – `POST /iserver/account/{accountId}/orders/whatif` or similar – preview margin impact and checks for an order without actually submitting (the docs mention "Preview Order / WhatIf Order"). This returns an estimation of whether the order would pass margin checks, etc.
- **Overnight Order** – possibly an endpoint to allow submitting after-hours for next day (not sure, might be for certain combos).
- **Bracket Orders & OCA** – The documentation has a section, meaning you can submit multiple orders in one array (the primary, take-profit, stop-loss) and IB will link them as a bracket or One-Cancels-All group if certain fields are set. Usually in the order JSON you'd use `isBracket` or send an array of orders with a `group` or `oca_group` specified. The doc likely explains how to do bracket orders in one call (the mention indicates it's supported).
- **Combo/Spread Orders** – as covered, use `conidex` field in place of conid, with the encoding of multiple legs. Otherwise similar to normal orders.
- **Cancel Order** – `DELETE /iserver/account/{accountId}/order/{orderId}` – cancel an order.
- **Modify Order** – `POST /iserver/account/{accountId}/order/{orderId}` – modify an open order.
- **Suppress Messages** – `POST /iserver/questions/suppress` – suppress certain order warnings (with body listing `messageIds`).
- **List Suppressible MessageIds** – The nav shows "Suppressible MessageIds", perhaps a reference section listing all possible messageId codes (like "o163", etc., what each means). Possibly a GET endpoint exists to retrieve which messages can be suppressed in the current session, or documentation listing them.
- **Reset Suppressed Messages** – `POST /iserver/questions/suppress/reset` – re-enable all suppressed warnings.

Portfolio Endpoints

- **Portfolio Accounts** – `GET /portfolio/accounts` – (covered above) list accounts for which the user can see info.
- **Portfolio Subaccounts** – `GET /portfolio/subaccounts` – (covered above) list subaccounts for advisors/brokers.
- **Portfolio Positions** – `GET /portfolio/{accountId}/positions` – get current positions for the account. (Nav shows "Positions" and "Positions (NEW)" perhaps indicating an updated version. Possibly `/portfolio/{accountId}/positions` and `/portfolio/{accountId}/positions/{conid}` etc.)
- **Combination Positions** – `GET /portfolio/{accountId}/positions/{conid}` might retrieve a breakdown of a combo position (the changelog mentions "Combination Position" endpoint added [268]).
- **Portfolio Allocation** – endpoints that show how a portfolio is allocated, perhaps across asset classes or sectors. Possibly "Portfolio Allocation (Single)" and "Portfolio Allocation (All)" were listed, meaning maybe something like `/portfolio/{accountId}/allocation` for a single account vs combined.

- **Portfolio Summary** – `GET /portfolio/{accountId}/summary` – returns account balances, margin info (as shown above).
- **Portfolio Ledger** – `GET /portfolio/{accountId}/ledger` – returns cash balances by currency (shown above).
- **Invalidate Portfolio Cache** – possibly an endpoint to invalidate server-side cache of portfolio data (maybe `POST /portfolio/{accountId}/positions/invalidate` or similar) to force fresh data.
- **Position & Contract Info** – Possibly an endpoint that given an account and a conid returns the position combined with contract details.

PortfolioAnalyst Endpoints (Performance Analysis)

These endpoints start with `/pa/` and require the PortfolioAnalyst permission. They allow retrieving performance data:

- **All Periods** – `POST /pa/analysis` (just guessing) or similar to get performance for all time periods. However in nav "All Periods" suggests maybe an endpoint that lists all available performance report periods or so.
- **Account Performance** – `POST /pa/performance` – likely returns performance metrics (returns, NAV history) for given account and timeframe. (The rate limit table shows /pa/performance, /pa/summary, /pa/transactions all at 1/15min, meaning these calls are heavy and limited) [269].
- **Transaction History** – `POST /pa/transactions` – retrieve list of transactions in the account (trades, deposits, withdrawals, fees, etc.) for a given period.

Each requires a JSON body, probably including date range and possibly account or consolidation parameters. The output would be complex JSON with performance metrics.

Scanner Endpoints

- **Scanner Parameters** – `GET /iserver/scanner/params` – returns parameters for market scanners (like available scan criteria). (Limited to 1 call / 15min) [270].
- **Run Scanner** – `POST /iserver/scanner/run` – run a market scan with specified criteria (the body includes parameters like instrument type, filters, etc., as obtained from the params call). Returns a list of scan results (contracts matching criteria). Limited to 1 req/sec [270].
- **Historical Scanner (HMDS)** – Possibly something under `/hmds/scanner` if available, or maybe not. The nav shows "HMDS Market Scanner" separately, implying possibly an endpoint to run scanner on historical data (less likely) or a different backend.

Session Endpoints

These include:

- **Authentication Status** – `GET /iserver/auth/status` – check if session is authenticated (as discussed above). Returns status JSON.
- **Initialize Brokerage Session** – `POST /iserver/auth/ssodh/init` – explicitly start a trading session (for OAuth flows or if needed).
- **Initialize /hmds Endpoints** – possibly `POST /iserver/auth/status` with some flag or another endpoint to initialize the market data (hmds) session if needed. Actually, maybe after

connecting via OAuth, you need to initialize both trading (iserver) and historical data (hmds) sessions. The nav line "Initialize /hmds endpoints" suggests an endpoint to initialize the historical data session (maybe `GET /iserver/marketdata/enable` or similar). Possibly the first call to a historical data endpoint does it too.

- **Logout** – `GET /logout` or specifically `GET /iserver/auth/logout` – ends the session (the Client Portal API might have an explicit logout endpoint to terminate the session).
- **Ping the server** – could refer to `/tickle` (keep-alive) or an actual ping endpoint that returns something trivial. But likely `/tickle` is meant.
- **Re-authenticate Brokerage Session (Deprecated)** – The nav lists a deprecated way to re-auth (maybe older endpoint).
- **Validate SSO** – `GET /sso/validate` – check if the SSO token is still valid or something (1 req/min limit) [271] .

Watchlist Endpoints

These allow management of watchlists (market data watchlists that a user can create in TWS/Client Portal):

- **Create a Watchlist** – `POST /iserver/account/watchlists` (or similar) – create a new watchlist with given tickers.
- **Get All Watchlists** – `GET /iserver/account/watchlists` – retrieve list of watchlists and their IDs for the user.
- **Get Watchlist Information** – `GET /iserver/account/watchlist/{watchlistId}` – get details (symbols) in a specific watchlist.
- **Delete a Watchlist** – `DELETE /iserver/account/watchlist/{watchlistId}` – remove a watchlist.

(The nav lines under Watchlists indicate these operations.)

WebSockets API

The IBKR Web API also provides a WebSocket interface for receiving asynchronous updates (market data, account updates, order status, etc.) without polling. To use it, you need to establish a WebSocket connection after obtaining a session token.

Connection Guide

To connect via WebSocket, first ensure you have an authenticated session. Then:

- **Request Session Information:** Call an endpoint (likely `GET /iserver/reauthenticate` or similar) to get a session ID or token if needed. Actually, IB's newer API (Client Portal API) uses an endpoint `/iserver/stream/session` which returns a WebSocket connection ticket and the WebSocket URL. The nav "Request Session Information" might refer to that.
- **Retrieve the Session Token:** Possibly the above step provides a token or you might use the same login authentication.
- **Establishing the WebSocket with Client Portal Gateway:** If using the Gateway, you likely connect to `wss://localhost:5000/v1/api/ws` or similar. If using OAuth (direct to IBKR), you connect to IBKR's WebSocket server (e.g., `wss://api.ibkr.com/v1/api/ws`). The nav splits "with Client Portal Gateway" vs "with OAuth" meaning the connection details differ (likely domain/host and authentication method differ).

- **Establishing the WebSocket with OAuth:** If using direct (no gateway), one might need to include an OAuth token in the connection query or headers. Possibly IB uses the session cookie or an authorization header.

Once connected via WebSocket, you can start sending subscription commands and receiving messages.

Solicited and Unsolicited Messages

Solicited messages are those you specifically request (like subscribing to market data or account updates), whereas **unsolicited** are events the server pushes without explicit request (like order status updates whenever an order changes, or FYI notifications). The documentation likely notes that certain data will come automatically once you're connected and authenticated (for instance, any live orders and executions updates might stream without an explicit subscribe command because the server knows your session and will push them).

Subscribing to WebSocket Topics

To subscribe to a particular data stream or "topic", you send specific text messages on the WebSocket after connection. Examples of topics:

- **Account Summary** (topic "acc" maybe),
- **Account Ledger** ("acd" or similar),
- **Live Orders** ("sor" perhaps for stream of orders),
- **Trades**,
- **Market Data** ("smd" as used above for snapshot, but for streaming it's actually the same "smd+conid" on WS).
- **News**, **Scanner** topics, etc., if any.

From Part I, we saw an example: `"smd+<conid>+{...}"` subscribes to market data for that conid. Similarly:

- To subscribe to **live orders** updates, IB uses a topic likely `"sor"` (the changelog mentioned *"the* `sor` *websocket topic now includes a filters argument"* [272]). So sending `"sor+{ "filters": ["Filled"] }"` might subscribe to order status updates, for example.
- To subscribe to **account updates**, possibly `"ssa"` or something – but IB's docs likely provide exact topics. There might be a prefix for account summary, e.g., `"sac"` for account summary (just guessing).
- **Unsubscribe** by sending messages starting with "u" (like `umd` to unsubscribe market data, which we saw; similarly maybe `uor` to unsubscribe orders).

Exact topics as per nav: under WebSockets, it lists Account Operations and Market Data separately:

Account Operations (WebSocket)

- **Subscribe Account Summary** – likely command to subscribe to account summary updates.
- **Unsubscribe Account Summary** – command to stop account summary updates.
- **Subscribe Account Ledger** – subscribe to cash balance updates (if different from summary).
- **Unsubscribe Account Ledger** – stop those.

These would push updates whenever account values change (equity, margin, etc.) without polling.

Market Data (WebSocket)

- **Market Data Request** – as shown, `smd+<conid>+{ fields: [...] }` to get live quotes.
- **Cancel Market Data** – `umd+<conid>+{}` to cancel.
- **Historical Market Data Request** – perhaps `shd+<conid>+{ "period":"1 D", "bar":"5 mins", ...}` to stream historical data (though IB might not stream historical data, they usually provide it via REST only; but maybe there is a way to stream bars).
- **Cancel Historical Market Data** – corresponding unsubscribe.
- **Subscribe to BookTrader Price Ladder** – maybe a topic like `sbd+<conid>` (book depth) to get order book depth updates.
- **Cancel Price Ladder Subscription** – corresponding cancel for book data.

Miscellaneous Operations (WebSocket)

- **Exercise Options** – not sure how that is a WS topic; likely it's a request you send if you want to exercise options through API (like an action). But maybe IB allows sending certain commands via WS as well as REST. Possibly an example of an unsolicited message could be an options exercise notification. Or they allow a request via WS to trigger an exercise. Unclear.
- **Ping Session** – maybe sending a ping through WS to keep it alive or check connection.

Order & Position Operations (WebSocket)

- **Request Live Order Updates** – possibly `sor` topic subscribe (as described).
- **Cancel Live Order Updates** – unsubscribing the above.
- **Request Profit & Loss** – maybe `spl` or something to subscribe to real-time P&L for open positions.
- **Cancel Profit & Loss** – unsub.
- **Request Trades data** – subscribe to execution reports maybe.
- **Cancel Trades data** – unsub.

Unsolicited Messages

Unsolicited WS messages include things like:

- **Account Updates** – If you're subscribed or by default, you might receive account updates. Possibly IB sends an unsolicited account update on initial connection with current values.
- **Order Status** – As orders are submitted/filled/canceled, messages come through (after subscribing to live orders topic). Possibly IB sends some by default too after connection to sync your current state.
- **Position Changes** – If subscribed, you'd get position change updates as they happen (like fills affecting positions).

Also, IBKR may send keep-alive or heartbeat messages, and possibly a message when session is about to expire or logged out elsewhere (so client knows to reconnect).

(For the complete WebSocket usage, refer to IB's documentation. The above outline is based on typical usage and hints from the navigation and changelog. The changelog mentioned `sor` *(stream orders) and an update to it, so that confirms some naming.)*

Part IV: Web API Changelog

(This section lists notable updates, new features, and changes to the Web API documentation and functionality over time, with the latest entries first. It covers changes in IBKR's API from 2024–2025 as given on the IBKR Campus changelog page [273] [274].)

- **December 10, 2025 –** *Update*: **IBKR WebAPI – Web API Reference** – The `/iserver/marketdata/snapshot` endpoint's `conids` parameter is now limited to **100 conids per query**, with a maximum of 50 field tags at a time [274]. (Previously, larger requests may have been allowed; now capped for performance.)

- **November 18, 2025 –** *Update*: **IBKR WebAPI – Trading Web API** – The `/hmds/history` endpoint was removed from documentation (deprecated). For historical data, use `/iserver/marketdata/history` instead [275]. (Clients should migrate any use of the old endpoint to the new one.)

- **August 27, 2025 –** *New*: **IBKR WebAPI – Trading Web API** – The WebSocket topic for live orders `sor` now supports a `"filters"` argument to filter orders by status [272]. (You can specify one order status to see only orders of that status through the feed.)

- **March 28, 2025 –** *Important*: **IBKR APIs – IBKR WebAPI – Web API Reference** – Effective May 1, all requests to the **Place Order**, **Cancel Order**, or **Modify Order** endpoints for **US Futures** must include the field `manualIndicator` [276]. This is to comply with CME Group Rule 536.B [277], distinguishing manual vs automated order origin. The `manualIndicator` should be set appropriately (likely true/false or 0/1) in the JSON when placing, modifying, or canceling futures orders.

- **December 18, 2024 –** *New*: **Account Management API** – The endpoint for account management requests was extended to support **assignment of fee template** and account **closure requests** [278]. (This pertains to IAM or account setup API, not the trading API directly.)

- **December 18, 2024 –** *New*: **Account Management API** – New types added for the `/enumerations` endpoint [279]:

- `COMPLEX_ASSET_TRANSFER` – query broker names for complex asset transfers (Basic FOP).
- `questionnaires` – get questions for Enhanced Due Diligence or Additional Verification tasks.
- `quiz-questions` – list of questions for IBKR knowledge assessment.
- `security-questions` – list of supported security questions for IBKR.

- **November 11, 2024 –** *New*: **IBKR API Changelog** – IBKR introduced separate changelog pages for each major API section [280]. (E.g., Web API has its own page, TWS API has its own, etc., to better organize updates.)

- **October 15, 2024 –** *Update*: **IBKR WebAPI – Trading Web API** – The **OAuth 1.0a documentation** was completely revamped [281]. It now clarifies all required RFC parameters and provides more examples for implementing OAuth 1.0a. The new structure is more procedural and provides exact information per endpoint (making it easier to implement step-by-step).

- **October 11, 2024 –** *Update*: **IBKR WebAPI** – The **Flex Web Service documentation** was fully updated to reflect the new Client Portal layout and updated URLs for `/SendRequest` and `/GetStatement` endpoints [282]. (If using Flex reports, note the documentation changes and the primary request URL changes.)

- **September 24, 2024 –** *Update*: **IBKR WebAPI – Trading Web API** – The **Live Orders endpoint** documentation was updated to describe the filter behavior [283]. (Likely referring to the query param `filters` which can filter by order status, as mentioned later with the `sor` update.)

- **September 10, 2024 –** *Update*: **IBKR WebAPI** – Updated Flex Web Service URLs for generating and retrieving reports [284]. (Ensuring the documentation uses the correct base URLs for Flex endpoints after system changes.)

- **September 6, 2024 –** *Important*: **IBKR WebAPI – Trading Web API** – The **Web API v1 Pacing Limitations section** was updated to better reflect global limits, and the list of endpoints was alphabetized for easier reference [285]. (This likely is the update from 10 req/sec to emphasizing it, or just reordering the documentation for clarity.)

- **August 30, 2024 –** *Update*: **IBKR WebAPI – Trading Web API** – The **Order Status documentation** was updated to include possible `503` error scenarios [286]. (IB added notes that certain order status requests might return 503 if e.g. backend is unavailable or session issues. Ensure your app handles HTTP 503 on order status checks.)

- **July 23, 2024 –** *New*: **Trading Web API** – Added the **Combination Position endpoint** to the Client Portal API documentation [268]. This endpoint allows retrieval of combo/spread positions held in the account as structured data (likely returning an object detailing each leg of combo positions).

- **July 23, 2024 –** *Update*: **Trading Web API** – Updated the **Live Orders endpoint** to include `order_ref` descriptions in the response [287]. (The `order_ref` field, which is a user-assigned order reference/comment, is now documented or returned properly.)

- **July 23, 2024 –** *New*: **IBKR WebAPI – Trading Web API** – **Beta release of the Client Portal Gateway** was updated to the latest software bundle [288]. (A new version of the gateway was released; users should update to take advantage of improvements.)

- **April 10, 2024 –** *Important*: The IBKRCampus API documentation underwent structural changes [289]:

- The previous standalone Client Portal API documentation was integrated into a **Client Portal API v1** page [290] (which corresponds to Part III of this document).
- The OAuth 1.0a documentation was merged into the general Client Portal API v1 documentation [290].

- A split content structure was introduced: endpoints reference under **Web API Reference**, and workflow/instructions under **Web API Documentation** [291]. (This corresponds to how Part I and Part II of this document are separated.)

- **April 10, 2024 –** *Update*: **IBKR WebAPI – Trading Web API** – The Client Portal WebSocket topic `spl+{}` (perhaps for P&L streaming) was changed to return `"uel"` instead of `"el"`. (This suggests a change in a JSON field name: maybe "unrealized equity loss" field or similar was

renamed from "el" to "uel". Specifically, it says *"spl+{} now returns 'uel' instead of 'el'"*. This likely affects subscribers to that topic; code should expect the new field name.)

- **March 7, 2024 –** *New*: **IBKR WebAPI – Trading Web API** – Added the endpoints `/iserver/questions/suppress` and `/iserver/questions/suppress/reset` (and possibly `/iserver/reply/{replyId}`) [292]. This introduced the ability to suppress order precautionary messages (as detailed in Order Reply Suppression above).

- **February 15, 2024 –** *Warning*: **IBKR WebAPI – Trading Web API** – Announced a change in how bid/ask sizes are formatted in the WebSocket **"sbd" (book data) subscription** by end of Feb 2024 [293]. Going forward, bid and ask sizes would be sent as raw unformatted numbers (no punctuation) instead of formatted strings. (Clients consuming order book data via WebSockets should adjust parsing accordingly.)

(Additional historical entries prior to 2024 are not listed here for brevity. For a full changelog, refer to IBKR's API changelog page. The above captures the key changes in 2024-2025 relevant to the Web API.)

Sources: All content compiled from Interactive Brokers API documentation pages on the IBKR Campus website [1] [172] and related reference materials.

---

[1] [2] [3] [4] [5] [6] [7] [8] [9] [10] [11] [12] [13] [14] [15] [16] [17] [18] [19] [20] [21] [22] [23] [24] [25] [26] [27] [28] [29] [30] [31] [32] [33] [34] [35] [36] [37] [38] [39] [40] [41] [42] [43] [44] [45] [46] [47] [48] [49] [50] [51] [52] [53] [54] [55] [56] [57] [58] [59] [60] [61] [62] [63] [64] [65] [66] [67] [68] [69] [70] [71] [72] [73] [74] [75] [76] [77] [78] [79] [80] [81] [82] [83] [84] [85] [86] [87] [88] [89] [90] [91] [92] [93] [94] [95] [96] [97] [98] [99] [100] [101] [102] [103] [104] [105] [106] [107] [108] [109] [110] [111] [112] [113] [114] [115] [116] [117] [118] [119] [120] [121] [122] [123] [124] [125] [126] Web API Documentation | IBKR API | IBKR Campus
https://www.interactivebrokers.com/campus/ibkr-api-page/webapi-doc/

[127] [128] [129] [130] [131] [132] [133] [134] [135] [136] [137] [138] [139] [140] [141] [142] [143] [144] [145] [146] [147] [148] [149] [150] [151] [152] [153] [154] [155] [156] [157] [158] [159] [160] [161] [162] [163] [164] [165] [166] [167] [170] [171] [172] [173] [174] [175] [176] [177] [178] [179] [180] [181] [182] [183] [184] [185] [186] [187] [188] [189] [190] [191] [192] [193] [194] [195] [196] [197] [198] [199] [200] [201] [202] [203] [204] [205] [206] [207] [208] [209] [210] [211] [212] [213] [214] [215] [216] [217] [218] [219] [220] [221] [222] [223] [224] [225] [226] [227] [228] [229] [230] [231] [232] [233] [234] [235] [236] [237] [238] [239] [240] [241] [242] [243] [244] [245] [246] [247] [248] [249] [250] [251] [252] [253] [254] [255] [256] [257] [258] [259] [260] [261] [262] [263] [264] [265] [266] [267] [269] [270] [271] Web API v1.0 Documentation | IBKR API | IBKR Campus
https://www.interactivebrokers.com/campus/ibkr-api-page/cpapi-v1/

[168] [169] Web API Reference | IBKR API | IBKR Campus
https://www.interactivebrokers.com/campus/ibkr-api-page/webapi-ref/

[268] [272] [273] [274] [275] [276] [277] [278] [279] [280] [281] [282] [283] [284] [285] [286] [287] [288] [289] [290] [291] [292] [293] Web API Changelog | IBKR API | IBKR Campus
https://www.interactivebrokers.com/campus/ibkr-api-page/web-api-changelog/